

7.Functional Languages

- iii-Strict and lazy Evaluation

CO523

Lec10-1



Strict evaluation (eager) computes expressions immediately when bound, while lazy evaluation delays evaluation until the value is required

- Strict (e.g., C, Java) offers predictable performance and debugging, whereas lazy (e.g., Haskell) enables infinite data structures, better optimization, and avoids unnecessary computations.

Key Differences:

- Evaluation Timing: Strict evaluates arguments before function calls; Lazy evaluates them only when needed.
- Performance: Strict can be faster for mandatory computations, while lazy avoids overhead for unused code.
- Memory: Lazy evaluation can be more memory-efficient by discarding intermediate results but may suffer from memory leaks due to "thunks" (unevaluated expressions).
- Use Cases: Use strict for predictable, imperative-style code; use lazy for infinite lists or when only partial data results are needed.

- Examples:
- Strict: $x = 2 + 2$ calculates 4 immediately.
- Lazy: $x = 2 + 2$ creates a thunk, calculating 4 only when x is used.

Lazy evaluation is a strategy for actually evaluating programs, and eagerness is the opposite strategy

why do we want lazy evaluation

How to analyze the performance of lazy algorithms?

What are some typical use cases of lazy evaluation ?

- Lazy evaluation is also critical for a lot of control structures, in any language. For example, if/then/else wouldn't be very useful if the then and else parts were always evaluated before you could even start executing the conditional selection logic.

How to analyze the performance of lazy algorithms ?

- The **Banker's Method**.

What are some typical use cases of lazy evaluation ?